



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 09.2

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 28 de abril del 2026

CALIFICACIÓN: _____

ACTIVIDADES REALIZADAS

EJERCICIO 1. Numero cambiante

Instrucciones: “Agregar que el número cambiante sea a una velocidad visible.”

MAIN

Primeramente, teníamos que declarar las variables (globales) con las que podríamos controlar esta velocidad, se opto por un acumulador y un tiempo de espera estático

```
// --- Animación: Control de Texturas (Offsets y Frames) ---
float AcumuladorTN;           // Cambio de frame en la textura de números
float tiempoDeEsperaN;        // Límite de tiempo para el cambio perceptible
...
...
// --- Animación: Control de Texturas (Offsets y Frames) ---
AcumuladorTN = 0.0f;
tiempoDeEsperaN = 25.0f;      // VELOCIDAD perceptible para el cambio
```

Después se modifico el numero animado, ligándolo al deltaTime y no enfocándonos en sumar solo valores al offset, dentro de nuestro bucle por cada ciclo de reloj, en este se cuenta si pasaron las 25.0f unidades para dar el cambio mediante nuestro acumulador y cuando así sucede se implementa el cambio.

```
// 8. NUMERO ANIMADO POR TIEMPO DELTATIME (Spritesheet Lógico) -----
AcumuladorTN += deltaTime; // Sumamos el tiempo real transcurrido

// Si alcanzamos las 25.0f unidades (tiempoDeEsperaN), saltamos al siguiente número
if (AcumuladorTN >= tiempoDeEsperaN) {
    toffsetnumerocambiau += 0.25f; // Cambiamos al siguiente número (offset de 0.25)
    if (toffsetnumerocambiau >= 1.0f) {
        toffsetnumerocambiau = 0.0f; // Reiniciamos si llegamos al final
    }
    AcumuladorTN = 0.0f; // Reseteamos el reloj
}

toffsetnumerov = 0.0;
toffset = glm::vec2(toffsetnumerocambiau, toffsetnumerov);
```

EJERCICIO 2 y 3. Animación Aelopile

Instrucciones: “Giro del Aelopile al prender el fuego y se detiene cuando el fuego se extingue. El Fuego animado, se prende con tecla, se "mueve" y se extingue.”

Mencionemos primero que ya se tenía el fuego a usar con el canal alfa y la transparencia activada desde el previo anterior.

Window .h y .cpp

Para window.h se agregó su respectivo getter además de un setter para identificar el estado de la bandera, mientras que en window.cpp además de iniciarlo en false, se colocó la letra L como su cambio en la función maneja teclado.

```
// --- VARIABLES PARA EL FUEGO ---
bool getAnimacionFuego() { return animacionFuego; } // Getter para leer si está prendido
void setAnimacionFuego(bool estado) { animacionFuego = estado; }
...
...
// [L]: Interruptor para encender/apagar el fuego animado
if (key == GLFW_KEY_L && action == GLFW_PRESS) {
    theWindow->animacionFuego = !theWindow->animacionFuego;
}
```

MAIN.CPP

Declaramos instanciamos variables de control globales fuera del ciclo principal, organizadas en dos partes (realmente 3, pero las relacionadas a los números ya se explicaron). El sistema de calentamiento nos ayuda a gestionar el movimiento del Aeolipile controlando mediante cronómetros los tiempos de calentamiento, la aceleración del ángulo de rotación y la disminución del mismo para simular un frenado. También tenemos las variables para el sistema visual para el fuego, estas nos almacenan los factores de escala necesarios para simular el escalamiento del fuego, así como su reducción de escala progresiva durante la fase de extinción del mismo.

```
// --- Animación: Giro del Aeolipile ---
float tiempoFuego; // Controlador de etapas para el fuego
float velAeolipile; // Velocidad de giro (Incrementa y baja)
float anguloAeolipile; // Rotación acumulada en grados (Para aplicar en Z)
float tiempoGirando; // Tiempo que lleva rotando (Al calentarse)
float tiempoApagando; // Tiempo que lleva desde que el fuego comienza a apagarse
// --- Animación: Efectos Visuales del Fuego ---
float EscalamientoY; // Factor de escala en Y
float EscalamientoXZ; // Factor de escala en XZ
float escalaExtincion; // Multiplicador de tamaño (1.0 a 0.0) para encoger el fuego al
apagarse
...
...
// EJERCICIO 9.2: Animaciones con Texturas y Transformaciones
// --- Animación: Control de Texturas (Offsets y Frames) ---
AcumuladorTN = 0.0f;
tiempoDeEsperaN = 25.0f; // VELOCIDAD perceptible para el cambio
// --- Animación: Giro del Aeolipile ---
tiempoFuego = 0.0f; // Cronómetro para los 2.5 segundos aprox
velAeolipile = 0.0f; // Velocidad de giro (Incrementa y baja)
anguloAeolipile = 0.0f; // Rotación acumulada en grados (Para aplicar en Z)
tiempoGirando = 0.0f; // Tiempo que lleva rotando (Al calentarse)
tiempoApagando = 0.0f; // Tiempo que lleva desde que el fuego comienza a apagarse
// --- Animación: Efectos Visuales del Fuego ---
EscalamientoY = 0.0f; // Factor de escala en Y
EscalamientoXZ = 0.0f; // Factor de escala en XZ
escalaExtincion = 0.0f; // Multiplicador de tamaño (1.0 a 0.0) para encoger
```

Para la animación instanciada dentro de cada ciclo de reloj, intentamos simular de forma algo más realista el comportamiento físico del Aeolipile, el cual se entiende que comienza a rotar mientras el agua comienza a hervir, se diseñó un sistema cinemático dividido en tres fases temporales controladas por el acumulador tiempoFuego. La primera etapa nos ayuda a poner un retardo que simula el tiempo necesario para que el agua comience a calentarse, manteniendo nuestro modelo sin rotación alguna. Pasada esta etapa después de 125.0f unidades, inicia la fase de aceleración, donde la velocidad de rotación incrementa gradualmente respecto al tiempo hasta alcanzar un límite de seguridad máximo de 15.0f. La tercera etapa inicia la extinción del fuego, el sistema pasa a una etapa de desaceleración que aplica una reducción en la rotación de forma continua.

```
// =====
// 6. Lógica de Rotación del Aeolipile -----
if (mainWindow.getAnimacionFuego()) {
    tiempoFuego += deltaTime; // Inicia el cronómetro del fuego
    // Inicia después de 125.0f (2.5 seg?)
    //      -> Acelera hasta llegar al tope en 625.f (+10 seg?)
    //      -> Se queda en el tope 875.0f (+5 seg?)
    if (tiempoFuego >= 125.0f && tiempoFuego < 875.0f) {
        tiempoGirando = tiempoFuego - 125.0f;
        // ARRANQUE SUAVE: La velocidad sube
        velAeolipile = tiempoGirando * 0.03f;
        // TOPE DE SEGURIDAD
        if (velAeolipile > 15.0f) {
            velAeolipile = 15.0f;
        }
    }
    // ¿15 seg? después de girar, comienza a relentizarse
    else if (tiempoFuego >= 875.0f) {
        tiempoApagando = tiempoFuego - 875.0f;
        // FRENADO: -0.06f (se detiene en 5 seg?)
        velAeolipile = 15.0f - (tiempoApagando * 0.06f);
        // Tope para evitar que gire en reversa al acabarse el fuego
        if (velAeolipile < 0.0f) {
            velAeolipile = 0.0f;
        }
    }
}
else {
    // Si apagamos el fuego, todo regresa a cero
    tiempoFuego = 0.0f;
    velAeolipile = 0.0f;
}
// Sumamos la velocidad actual
anguloAeolipile += velAeolipile * deltaTime;
// Reset para no llegar a numeros gigantes
if (anguloAeolipile > 360.0f) anguloAeolipile -= 360.0f;
// =====
```

El único cambio en el Aeolipile se aplico en la rotación animada al instanciar el modelo opaco.

```
// =====
// AEOLIPILE -----
// BASE
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, -2.0f, 20.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Aeolipile_base_M.RenderModel();
// CIRCULO
```

```

model = glm::translate(model, glm::vec3(0.0f, 4.8f, 0.0f));
// --- APLICAMOS LA ROTACIÓN ANIMADA ---
model = glm::rotate(model, anguloAeolipile * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Aeolipile_M.RenderModel();
// =====

```

Para la animación del fuego, se implementaron dos efectos visuales simultáneos a través de la manipulación de la escala del fuego, el escalamiento se logró aplicando funciones trigonométricas (sin y cos) dependientes de nuestro tiempo continuo que ya hemos instanciado antes como now, esta escala se aplicó sobre los 3 ejes del fuego, para el cual es importante señalar que se instancio como el agave.

Paralelamente, la lógica de extinción de fuego evalúa el cronómetro global que instanciamos, (tiempoFuego) y al superar el tiempo que establecimos (875.0 unidades), calculamos una reducción progresiva a nuestro multiplicador global (escalaExtincion) de 1.0 a 0.0 a lo largo de 5 segundos. Ambos factores de escala se multiplican antes de aplicarse a la matriz del modelo. Una vez que la escala final llega a cero, haciendo que el fuego no se vea más, reseteamos la bandera de la máquina de estados desde el propio código y ocultando la malla sin requerir intervención forzosa del teclado.

```

// =====
// 10. FUEGO ANIMADO -----
if (mainWindow.getAnimacionFuego()) {
    model = glm::mat4(1.0f);
    model = glm::translate(model, glm::vec3(0.0f, -0.7f, 20.0f));

    // --- LÓGICA DE EXTINCIÓN ---
    escalaExtincion = 1.0f; // 1.0 significa 100% de tamaño normal
    // Si ya se alcanza el giro max y los ¿5 seg? extra (625 + 250 = 875.0f)
    if (tiempoFuego > 875.0f) {
        // Tiempo para apagar (empieza en 0)
        tiempoApagando = tiempoFuego - 875.0f;
        // En 5 segundos (250.0f) debe llegar a 0.0
        escalaExtincion = 1.0f - (tiempoApagando / 250.0f);
        // Tope final: Cuando se encoge por completo, lo apagamos del sistema
        if (escalaExtincion <= 0.0f) {
            escalaExtincion = 0.0f;
            mainWindow.setAnimacionFuego(false); // False para usar 'L' nuevamente
        }
    }

    // --- ANIMACIÓN DE ESCALADO ---
    EscalamientoY = 1.0f + (sin(now * 15.0f) * 0.15f); // Estira hacia arriba/abajo
    EscalamientoXZ = 1.0f + (cos(now * 12.0f) * 0.05f); // Engorda un poquito a los lados

    // Aplicamos la escala dinámica Y LA MULTIPLICAMOS POR LA EXTINCIÓN
    model = glm::scale(model, glm::vec3(EscalamientoXZ * escalaExtincion, EscalamientoY *
    * escalaExtincion, EscalamientoXZ * escalaExtincion));
    model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));

    // Limpiamos los offsets
    toffset = glm::vec2(0.0f, 0.0f);
    glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
    color = glm::vec3(1.0f, 1.0f, 1.0f);
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    Fuego.UseTexture();
    Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
    meshList[3]->RenderMesh(); //Mesh 3 (Planos cruzados en X como el agave)
}
// =====

```

EJECUCION

Se hace uso de la letra L para iniciar el fuego.



Se adjunta el link del VIDEO en drive.

https://drive.google.com/file/d/1-vYHLptOE2gEPrEoPJJoosvKRCN_Q4t_5/view?usp=sharing

PROBLEMAS PRESENTADOS

- **Problema:** Al intentar programar la animación completa del sistema (fuego escalado, cronómetros de retardo, aceleración del Aeolipile y desaceleración simultánea) en un solo bloque de código, resultaba algo difícil de comprender en cuanto a la lógica matemática entera. Los errores de sintaxis y los fallos físicos (como giros inversos o aceleraciones infinitas) eran difíciles de identificar, lo que hizo que el problema llevara más tiempo de resolver que lo usual.
- **Solución:** Se prefirió empezar de cero haciendo cada parte de la animación por etapas. En lugar de codificar todo el algoritmo de golpe, el desarrollo se dividió en fases pequeñas para asimilar el funcionamiento de cada concepto. Primero se

implementó la aparición estática del fuego; luego se añadió su escalamiento en sus ejes; posteriormente se introdujo la variable `deltaTime` para crear el cronómetro independiente, y solo hasta comprender ese paso, se acopló la física de aceleración y detenimiento del Aeolipile. Este enfoque paso a paso permitió entender a fondo el impacto de cada variable antes de integrarla a nuestro sistema principal, garantizando un control total sobre la animación final.

CONCLUSION

Sobre los ejercicios de clase

La realización de este ejercicio fue sin duda mucho más sencilla que la práctica anterior. La integración de la variable `deltaTime` fue el concepto más importante, ya que permitió desvincular el movimiento de la tasa de fotogramas de nuestro procesador, garantizando consistencia en cualquier hardware. Asimismo, el uso de funciones trigonométricas (`sin` y `cos`) demostró cómo las matemáticas puras son la herramienta principal para dotar a los modelos 3D de un comportamiento orgánico, menos estático y más pulido, elevando la calidad técnica de nuestro entorno gráfico.

Comentarios generales

Para este ejercicio ya todo en su mayoría es entendible, aunque se hubiera preferido dar más énfasis a las variables usadas en la lógica de animación o a más fórmulas específicas para poder aplicarlas en mejores movimientos de nuestro modelo.

REFERENCIAS

- Quilez, I. (2010). *GraphToy* (Versión 0.4) [Aplicación web]. Recuperado de <https://graphtoy.com/>